

UNIVERSIDADE DO ESTADO DA BAHIA
DEPARTAMENTO DE CIÊNCIAS EXATAS E DA TERRA - I
COLEGIADO DO CURSO DE SISTEMAS DA INFORMAÇÃO

ALANA CAROLINA PEREIRA ABREU
MATEUS DOS SANTOS OLIVEIRA
RAFAEL DO NASCIMENTO ARGÔLO
TATIANA RIBEIRO OLIVEIRA

**RELATÓRIO DO TRABALHO 4: Controle de bateria em
microgrid com energia solar**

SALVADOR

2026

1. Definição do problema

A integração de fontes renováveis de geração distribuída, como a energia solar fotovoltaica, em microrredes elétricas apresenta um desafio relevante: a geração solar é variável e nem sempre coincide com os períodos de maior demanda. A presença de uma bateria e a possibilidade de comprar ou vender energia para a rede aumenta o número de decisões operacionais possíveis.

O problema consiste em determinar, a cada hora, a ação mais adequada para o sistema: manter o estado da bateria, carregar usando excedente solar, descarregar para reduzir a compra da rede ou vender o excedente solar disponível.

O objetivo é minimizar o custo líquido de energia, evitar movimentações desnecessárias da bateria e impedir ações operacionalmente inválidas. Devem ser respeitadas restrições como capacidade máxima, potência de carga e descarga, eficiências e estado de carga mínimo.

Na simplificação adotada, a rede elétrica permanece sempre disponível e possui capacidade suficiente para suprir qualquer déficit. Portanto, toda a demanda é atendida. A dificuldade do problema está em decidir quando depender da rede, quando preservar energia na bateria e quando aproveitar o excedente solar.

A demanda, a geração solar, o SOC inicial e os preços variam entre episódios. Dessa forma, o problema constitui uma decisão sequencial sob incerteza e pode ser aproximado por um Processo de Decisão de Markov.

2. Justificativa do enquadramento do problema em AR (Aprend. Refor.)

A operação da microgrid é um problema sequencial porque cada decisão modifica as possibilidades futuras. Carregar a bateria durante o período solar pode gerar um pequeno custo de degradação no presente, mas permitir economia no horário de ponta. Da mesma forma, descarregar cedo pode reduzir o custo imediato e deixar pouca energia disponível para um momento mais caro.

DQN e PPO são algoritmos model-free: eles não precisam receber explicitamente uma função de transição que descreva todas as probabilidades do sistema. Entretanto, dependem da interação com o ambiente simulado para observar as consequências das ações e aprender uma política.

O problema reúne características relevantes para Aprendizado por Reforço:

- decisões sequenciais;
- recompensa atrasada;
- necessidade de planejamento por várias horas;
- demanda e geração solar estocásticas;
- recurso limitado pela capacidade da bateria;
- custo associado ao uso da bateria;
- ações que podem ser inválidas em determinados estados;
- conflito entre economia imediata e preservação de energia;
- dinâmica variável entre episódios.

3. Modelagem do ambiente

O ambiente de simulação foi modelado como um processo de decisão de Markov (MDP) e implementado utilizando a interface padrão da biblioteca Gymnasium, o que permite a utilização direta de algoritmos de Aprendizagem por Reforço disponíveis em frameworks consolidados.

- Estados

O vetor de observação é composto por sete variáveis contínuas, todas normalizadas para facilitar a convergência do processo de aprendizado:

Índice	Variável	Faixa	Descrição
0	SOC	[0, 1]	Estado de carga atual da bateria
1	Demanda	[0, 1]	Demanda normalizada em relação ao valor de referência (4,5 kWh)
2	Geração solar	[0, 1]	Geração solar normalizada em relação ao valor de referência (5,0 kWh)
3	Preço de compra	[0, 1]	Preço de compra normalizado em relação ao valor máximo

			(1,5 R\$/kWh)
4	Preço de venda	[0, 1]	Preço de venda normalizado em relação ao valor máximo
5	Seno da hora	[-1, 1]	Representação cíclica do horário do dia (seno)
6	Cosseno da hora	[-1, 1]	Representação cíclica do horário do dia (cosseno)

Os dois últimos componentes — seno e cosseno da hora — foram incluídos para representar o caráter cíclico do tempo ao longo do dia, evitando a descontinuidade que ocorreria se o horário fosse representado como um valor absoluto (por exemplo, a diferença entre 23h e 0h seria tratada como grande, quando na realidade são consecutivas).

- Ações

O espaço de ações é discreto e contém quatro opções:

Índice	Ação	Descrição
0	Manter	Utiliza solar para atender a demanda e compra da rede o déficit. A bateria não é movimentada e o excedente solar não é vendido.
1	Carregar	Utiliza excedente solar para carregar a bateria; compra da rede o déficit e vende o excedente que não couber na bateria.
2	Descarregar	Utiliza a bateria exclusivamente para reduzir o déficit da demanda; a rede compra o restante.
3	Vender	Vende o excedente solar disponível e compra da rede o déficit da demanda.

Nem todas as ações são viáveis em todos os estados. Por exemplo, a ação de carregar (1) só gera armazenamento efetivo quando há excedente solar disponível, e a ação de vender (3) é considerada inválida quando a geração solar é nula. Essas restrições são tratadas no mecanismo de recompensa, por meio de penalizações. Exemplo:

- A compra da rede não constitui uma ação separada. Ela ocorre automaticamente sempre que a geração solar e a descarga da bateria não forem suficientes.
- A ação de carregar é inválida quando não produz carga efetiva.

- A ação de descarregar é inválida quando não existe déficit ou energia disponível.
- A ação de vender é inválida quando não existe excedente solar.

- Recompensas

A recompensa recebida pelo agente em cada passo de tempo é calculada pela subtração dos custos e penalidades associados à operação da microgrid. Dessa forma, maximizar a recompensa acumulada ao longo do episódio equivale a minimizar o custo operacional total do sistema.

$$r_t = - (C_{energia} + C_{degradação} + P_{ação inválida})$$

Onde:

- **Custo de energia ($C_{energia}$):** diferença entre o custo da energia comprada da rede e a receita obtida com a venda do excedente de energia. Esse componente representa o custo financeiro direto da operação.
- **Custo de degradação da bateria ($C_{degradação}$):** produto da energia movimentada pela bateria (carga + descarga) pelo custo unitário de degradação (0,04 R\$/kWh). Esse termo penaliza ciclos desnecessários de carga e descarga, contribuindo para preservar a vida útil da bateria.
- **Penalidade por ação inválida ($P_{ação inválida}$):** penalidade fixa de 0,25 aplicada quando o agente executa uma ação incompatível com o estado atual do ambiente, incentivando o aprendizado das restrições operacionais da microgrid.

A recompensa pode ser positiva em uma hora na qual a receita da venda seja superior aos demais custos. No acumulado, maximizar a recompensa significa buscar uma política que combine:

- baixo custo líquido de energia;
- uso economicamente justificável da bateria;
- poucas ações inválidas;
- planejamento de longo prazo.

Ao maximizar a recompensa acumulada durante o treinamento, o agente aprende uma política capaz de reduzir os custos operacionais, preservar a bateria, evitar ações inválidas e atender à demanda energética, equilibrando benefícios imediatos e consequências futuras.

- Episódios

Cada episódio corresponde a um ciclo completo de 24 horas, com passos de tempo de 1 hora. Os perfis de demanda,

geração solar e preços de energia são gerados aleatoriamente a cada reinicialização do episódio, seguindo modelos estocásticos que representam padrões típicos de uma microgrid:

- **Demanda:** combinação de um componente base constante com dois picos gaussianos (um pela manhã e outro à noite), acrescida de ruído;
- **Geração solar:** padrão senoidal centrado ao meio-dia, modulado por um fator de cobertura de nuvens aleatório e ruído;
- **Preços:** três faixas de preço (off-peak, shoulder e peak) com variação aleatória de até 5%, além de um preço de venda proporcional ao preço de compra (45%).

O estado inicial da bateria (SOC) também é sorteado aleatoriamente, variando entre 25% e 75% da capacidade total.

- Critérios de término

O episódio se encerra quando o horário atinge o limite de 24 passos (terminação natural). Não há truncamento, ou seja, nenhum episódio é interrompido antes de seu ciclo completo. Ao final do episódio, o vetor de observação é retornado como zeros, sinalizando ao algoritmo de aprendizado que a sequência foi concluída.

4. Implementação do ambiente Gymnasium

A implementação Gymnasium do MicrogridEnv (`microgrid/envs/microgrid_env.py`) apresenta as seguintes características:

Estrutura geral: Herda de `gym.Env` e implementa todos os métodos obrigatórios (`reset()`, `step()`, `render()` e `close()`). Utiliza a dataclass **StepMetrics** para registrar detalhadamente as métricas de cada passo da simulação.

Observação (7 dimensões): O método `_observation()` monta um vetor normalizado contendo o estado de carga da bateria (SOC), demanda, geração solar, preço de compra e preço de venda da energia (todos normalizados com `np.clip()` e valores máximos de referência para permanecerem entre 0 e 1), além das componentes seno e cosseno do horário, formando uma representação **cíclica** do tempo e evitando a descontinuidade entre as horas 23 e 0.

reset(): Chama `super().reset(seed=seed)` para inicializar o

gerador de números aleatórios (RNG) do Gymnasium, gera os três perfis estocásticos (demanda, geração solar e preços) utilizando `self.np_random` (RNG controlado pelo Gymnasium) e inicializa aleatoriamente o estado de carga da bateria (SOC) entre **0,25** e **0,75**.

step() — o coração do ambiente: Implementa o despacho energético da microrrede. Inicialmente calcula quanto da geração solar atende diretamente à demanda (`solar_to_load = min(solar, demand)`). Em seguida, o fluxo energético é definido de acordo com a ação escolhida pelo agente:

- **Ação 0 (Manter):** Compra da rede toda a energia necessária para suprir o déficit de demanda.
- **Ação 1 (Carregar):** O excedente de geração solar é utilizado para carregar a bateria; caso ainda exista excedente, ele é vendido para a rede. O déficit de energia é suprido pela rede elétrica. Também possui lógica adicional para a opção **allow_grid_charging**.
- **Ação 2 (Descarregar):** a bateria fornece energia somente para reduzir o déficit da demanda. Caso a descarga não seja suficiente, a rede elétrica atende o déficit restante. Essa ação não vende automaticamente o excedente solar. Quando não existe déficit ou a bateria não pode fornecer energia, a ação é marcada como inválida.
- **Ação 3 (Vender):** Todo o excedente de geração solar é vendido para a rede elétrica, enquanto o déficit de demanda é atendido pela compra de energia da rede.

Reward function: combina três componentes: custo líquido de energia, degradação proporcional ao throughput da bateria e penalidade fixa por ação inválida. As ações inválidas não são bloqueadas pelo ambiente; elas apenas recebem uma penalização na recompensa.

Conformidade com o Gymnasium: Passa no `check_env()` do Stable-Baselines3, utiliza `self.np_random` para garantir sementes controladas, retorna corretamente a tupla (`observation`, `reward`, `terminated`, `truncated`, `info`) e define `terminated = True` quando `hour >= 24`.

Observação pós-terminal: Quando o episódio termina, a implementação retorna um vetor preenchido com zeros no lugar de uma nova observação, evitando o acesso a estados inexistentes após o encerramento do episódio.

5. Algoritmos usados

O projeto utiliza dois algoritmos da biblioteca **Stable-Baselines3**, ambos configurados com a política **MlpPolicy**, que emprega uma rede neural totalmente conectada (*fully connected*) para aproximar as funções de decisão.

DQN (Deep Q-Network)

Implementado no script `scripts/train_dqn.py`, o DQN é um algoritmo **baseado em valor (*value-based*)**, que aprende uma aproximação da função Q por meio de uma rede neural.

As principais configurações utilizadas foram:

- *Experience Replay* com buffer de **50.000 transições**;
- estratégia de exploração **ϵ -greedy**, com decaimento do ϵ de **1,0 para 0,05**, explorando aproximadamente **25%** do treinamento;
- atualização da **Target Network** a cada **1.000 passos**;
- tamanho do *batch* igual a **64**;
- *learning rate* de **0,001**;
- fator de desconto (γ) igual a **0,99**;
- treinamento durante **50.000 timesteps**.

PPO (Proximal Policy Optimization)

Implementado no script `scripts/train_ppo.py`, o PPO é um algoritmo **baseado em política (*policy-based*)**, que aprende diretamente a política de decisão do agente.

As principais configurações utilizadas foram:

- utilização do **Generalized Advantage Estimation (GAE)** com **$\lambda = 0,95$** ;
- mecanismo de **clipping** para limitar as atualizações da política e aumentar a estabilidade do treinamento;
- **10 épocas** de otimização para cada lote de dados;
- **Entropy Bonus** (coeficiente **0,01**) para incentivar a exploração;
- *learning rate* de **0,0003**;
- **512 passos** por *rollout* (`n_steps = 512`);
- treinamento durante **50.000 timesteps**.

Estratégia de treinamento e avaliação

Ambos os algoritmos foram treinados utilizando **cinco sementes aleatórias diferentes (10, 20, 30, 40 e 50)**, com o objetivo de reduzir a influência da aleatoriedade nos resultados.

Após o treinamento, os modelos foram avaliados em **30 a 50 episódios**, utilizando sementes que não haviam sido empregadas durante o treinamento, permitindo uma avaliação mais consistente da capacidade de generalização dos agentes.

Além disso, foram utilizados dois métodos de referência (**baselines**):

- **agente aleatório**, que seleciona ações de forma totalmente aleatória;
- **agente heurístico**, baseado em regras pré-definidas para o gerenciamento da microrrede.

Esses baselines foram utilizados como referência para comparar o desempenho obtido pelos algoritmos de Aprendizado por Reforço.

6. Descrição das baselines

RandomAgent (microgrid/baselines/random_agent.py): A cada passo, escolhe uniformemente entre as 4 ações. Não usa nenhuma informação do estado. Serve como piso mínimo de performance — qualquer coisa melhor que ele "aprendeu" algo.

HeuristicAgent (microgrid/baselines/heuristic_agent.py): Agente baseado em regras manuais usando as observações normalizadas:

- se a geração solar em kWh superar a demanda e o SOC for menor que 92%, carregar;
- se o preço de compra for maior ou igual a `1,05` e o SOC superar 25%, descarregar;
- se houver excedente solar e o SOC for maior ou igual a 92%, vender;
- nos demais estados, manter.

A heurística representa uma política manual razoável e funciona como referência econômica. Ela não deve ser descrita necessariamente como “teto”, pois um agente pode obter custo menor e ainda apresentar recompensa inferior devido à degradação e às ações inválidas.

É um "expert system" que tenta imitar uma estratégia lógica que um humano seguiria. Serve como teto de referência — se o RL superar ele, aprendeu uma estratégia melhor que a manual.

7. Estratégia de treinamento e avaliação

DQN e PPO foram treinados com cinco sementes: 10, 20, 30, 40 e 50.

Cada combinação de algoritmo e semente foi treinada durante 50 mil passos, totalizando:

$$2 \times 5 \times 50.000 = 500.000$$

passos de treinamento.

A avaliação utilizou exatamente 30 sementes não vistas: 3000–3029.

Cada um dos cinco modelos DQN e dos cinco modelos PPO foi avaliado nesses mesmos 30 cenários. A heurística e o agente aleatório também foram avaliados nas mesmas sementes, garantindo comparabilidade.

8. Critério de sucesso e falha

Cada episódio do agente é comparado com a heurística executada na mesma semente.

O regret relativo é:

$$\text{regret} = \frac{C_{\text{agente}} - C_{\text{heurística}}}{\max(|C_{\text{heurística}}|, 10^{-8})}$$

O limite variável de custo é:

$$C_{\text{limite}} = C_{\text{heurística}} + 0,20 \{|C_{\text{heurística}}|\}$$

Um episódio é classificado como falha quando:

- possui mais de três ações inválidas; ou
- apresenta custo superior ao limite variável.

Caso contrário, é considerado sucesso.

Um regret negativo significa que o agente foi mais barato que a heurística. Um regret positivo significa que foi mais caro.

9. Agregação estatística

Os episódios de DQN e PPO são resumidos em dois níveis:

1. calcula-se a média dos 30 episódios para cada semente de treinamento;
2. calculam-se média e desvio padrão entre as cinco sementes de treinamento.

Assim, o desvio principal de DQN e PPO representa sensibilidade ao processo de treinamento.

As baselines não possuem sementes de treinamento. Para elas, média e desvio são calculados entre os 30 cenários de avaliação. Essa diferença deve ser mencionada ao interpretar a tabela.

10. Protocolo experimental

Protocolo experimental (scripts/run_experiments.py):

Fase 1 — Treinamento: Cada algoritmo (DQN e PPO) é treinado 5 vezes com seeds diferentes (10, 20, 30, 40, 50), totalizando 10 modelos treinados. Cada treinamento roda 50K timesteps. Isso garante que os resultados não são fruto de uma seed específica.

Fase 2 — Avaliação: Todos os agentes são avaliados nos mesmos 30 episódios (seeds 3000-3030), incluindo:

- 5 modelos DQN (um por seed de treino)
- 5 modelos PPO (um por seed de treino)
- RandomAgent (seed 777 fixa para ações)
- HeuristicAgent (determinístico)

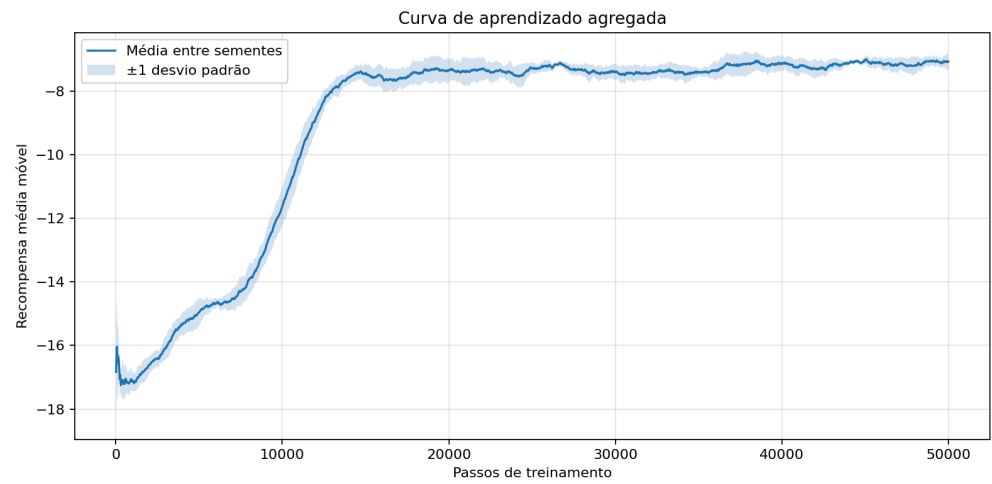
Separação treino/avaliação: Seeds de treino (10-50) e de avaliação (3000-3029) são completamente diferentes, garantindo que os agentes RL são testados em cenários que nunca viram durante o treinamento.

Métricas coletadas por episódio: Recompensa total, custo de energia, compra/venda da rede, throughput da bateria, demanda não atendida, ações inválidas, SOC final, e um indicador de "sucesso" (demanda não atendida ≈ 0 e < 3 ações inválidas).

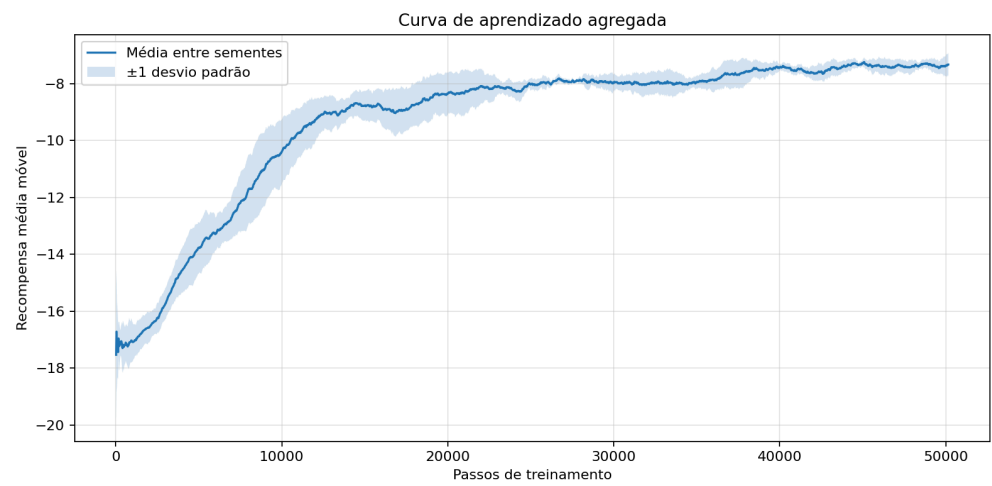
Resultados: Salvos em results/summaries/ como CSV — tanto os dados episódicos (all_experiment_episodes.csv) quanto o resumo agregado com média/desvio padrão por agente (all_experiment_summary.csv). A função aggregate_results agrupa por nome do agente e calcula mean/std em todas as métricas.

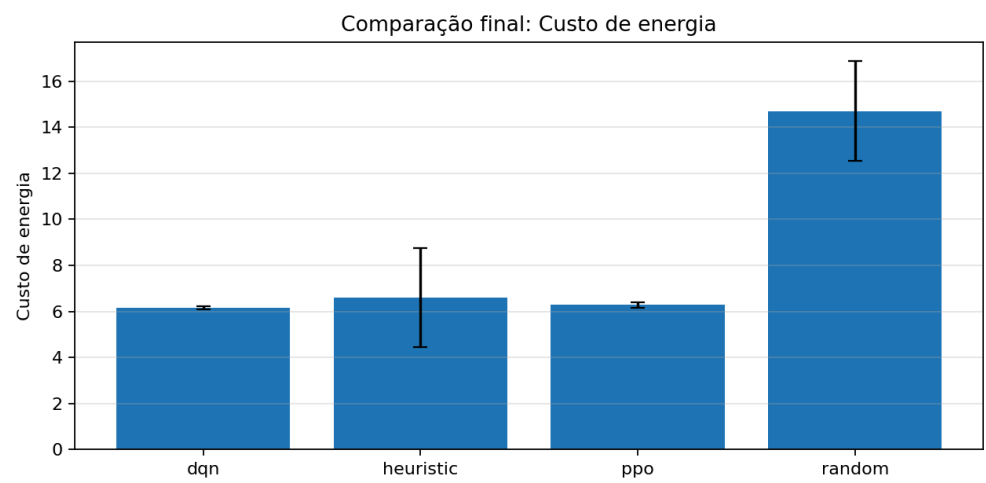
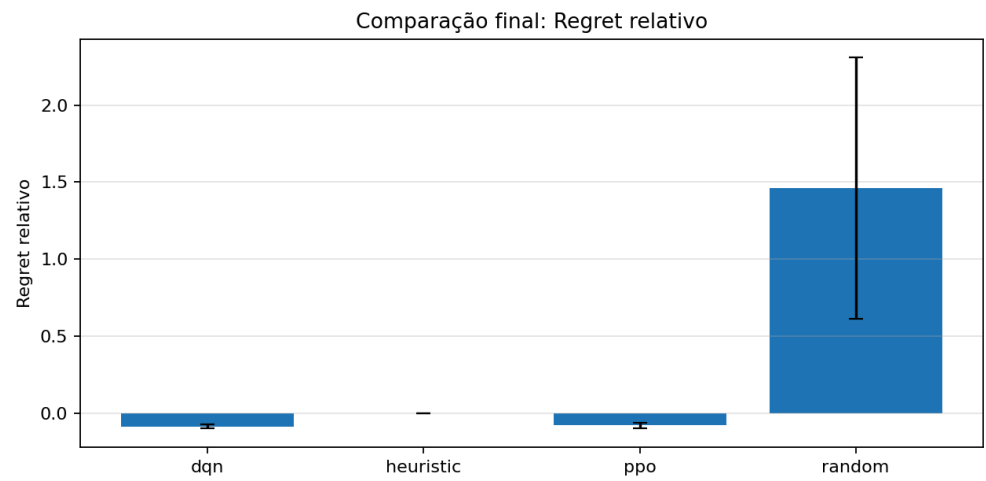
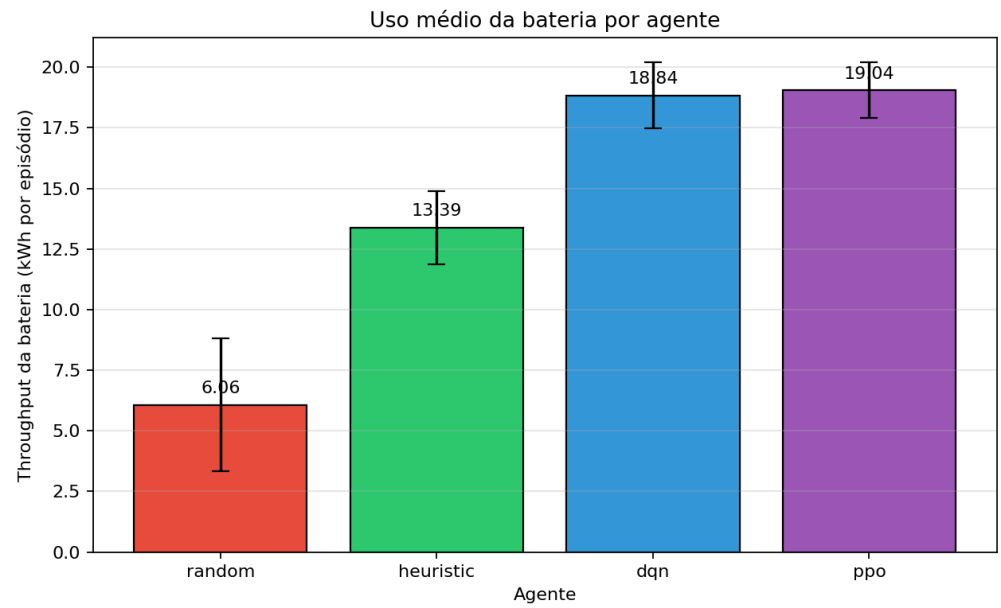
11. Resultados

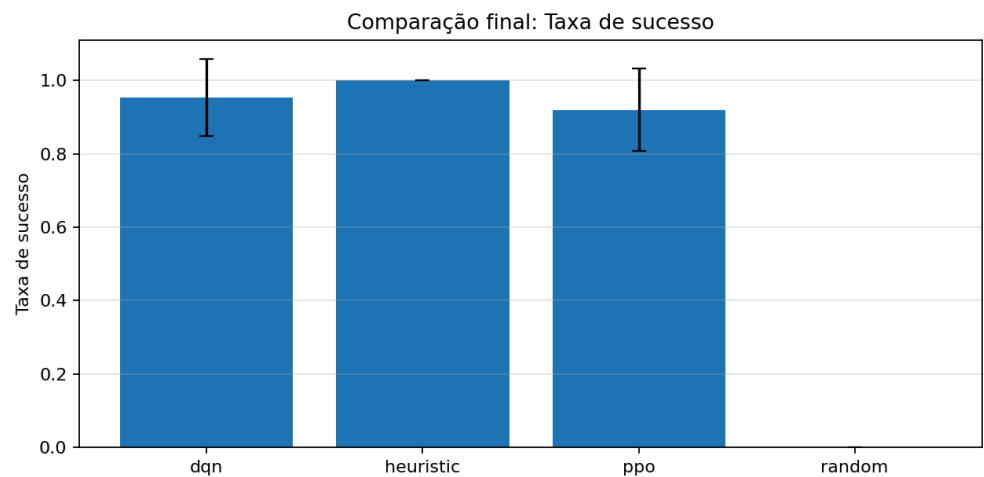
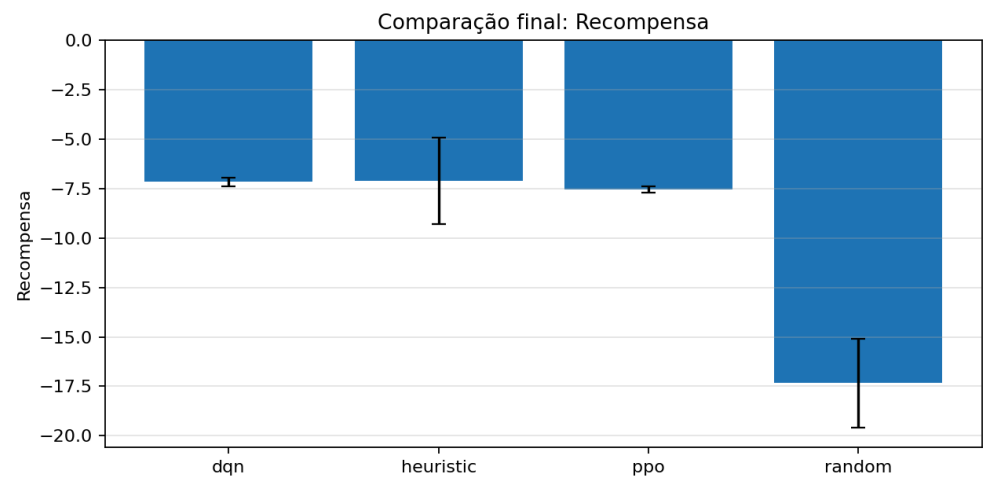
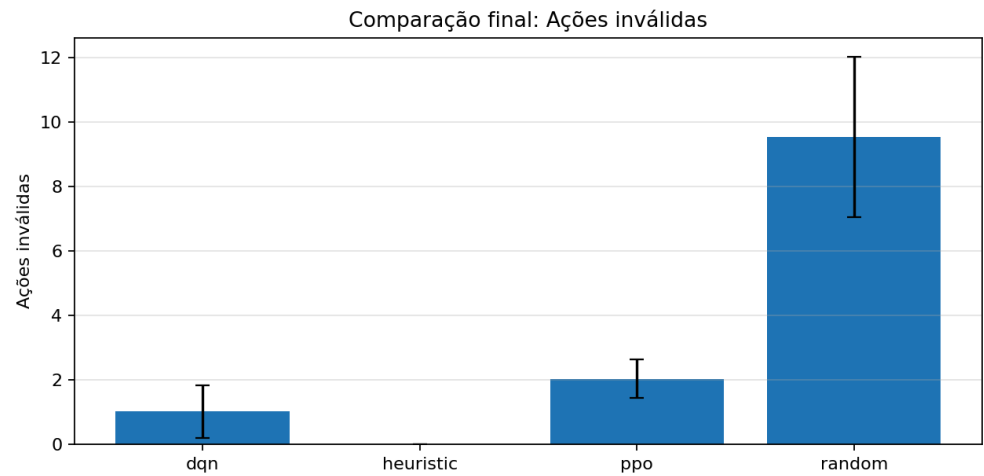
Curva de aprendizado do DQN:



Curva de aprendizado do PPO:







Em Aprendizado por Reforço, o agente aprende por interação. A cada hora, ele observa o estado da bateria, a demanda, a geração solar, os preços e o horário. Em seguida, escolhe entre

manter, carregar, descarregar ou vender. A consequência dessa ação produz uma recompensa.

Nossa recompensa não considera apenas o custo de energia. Ela também penaliza a degradação da bateria e as ações inválidas. Por isso, o agente com menor custo financeiro não necessariamente apresenta a maior recompensa.

O DQN obteve o menor custo, aproximadamente 6,16 unidades monetárias, contra 6,60 da heurística. Entretanto, ele movimentou cerca de 18,84 kWh pela bateria, enquanto a heurística movimentou 13,39 kWh. Além disso, o DQN executou aproximadamente uma ação inválida por episódio.

Quando somamos custo de energia, degradação e penalidades, a recompensa do DQN fica em aproximadamente -7,17. A heurística, apesar de ter custo de energia maior, não executa ações inválidas e utiliza menos a bateria, alcançando recompensa de -7,13.

O PPO apresentou uma estratégia ainda mais agressiva. Ele teve throughput próximo de 19 kWh e aproximadamente duas ações inválidas por episódio. Isso explica sua recompensa inferior, de aproximadamente -7,54.

O agente aleatório teve recompensa muito pior, custo superior e cerca de nove ações inválidas por episódio. Isso demonstra que DQN e PPO realmente aprenderam comportamento relevante.

Portanto, a heurística teve a melhor recompensa porque equilibrar melhor os três componentes do objetivo. Já o DQN foi o melhor agente treinado e conseguiu o menor custo financeiro. O resultado evidencia o conflito entre economizar energia e preservar a bateria.

12. Análise crítica

Os resultados demonstram que DQN e PPO aprenderam comportamento relevante. Ambos superaram amplamente o agente aleatório e reduziram o custo financeiro em relação à heurística.

Entretanto, menor custo não significa automaticamente maior recompensa. A recompensa também considera degradação e

ações inválidas. Por isso, a heurística obteve recompensa ligeiramente superior ao DQN, mesmo apresentando custo de energia maior.

O resultado evidencia o principal conflito do problema: usar intensamente a bateria reduz compras caras, mas aumenta o desgaste. O DQN apresentou o melhor equilíbrio entre os agentes treinados, enquanto o PPO adotou uma estratégia mais agressiva.

As taxas de sucesso de 95,3% para DQN e 92% para PPO indicam boa generalização, mas também mostram que algumas sementes ainda produzem episódios com custo elevado ou excesso de ações inválidas.

Não é possível concluir que DQN é universalmente superior ao PPO apenas pelas médias. As diferenças devem ser interpretadas junto com as curvas de treinamento, variação entre sementes e análise dos episódios de falha.

O projeto implementou um ambiente próprio compatível com Gymnasium e comparou duas técnicas de Aprendizado por Reforço, DQN e PPO, com baselines aleatória e heurística.

Os agentes treinados aprenderam a reduzir o custo e a compra da rede. O DQN apresentou o melhor equilíbrio entre economia, estabilidade e taxa de sucesso entre os agentes de AR. O PPO apresentou uma política mais agressiva, com maior uso da bateria e mais ações inválidas.

A heurística permaneceu superior em recompensa total por operar sem ações inválidas e com menor degradação. O agente aleatório teve desempenho substancialmente inferior, confirmando que DQN e PPO aprenderam políticas relevantes.

O principal resultado é a identificação de um trade-off: reduzir o custo financeiro exige utilizar mais intensamente a bateria, o que aumenta a degradação e pode diminuir a recompensa total.

13. Limitações e melhorias futuras

Limitações:

- Rede ilimitada: não existem falhas ou limites de potência da rede.

- Ações discretas: o agente escolhe o tipo de operação, mas não controla diretamente uma potência contínua.
- Dados sintéticos: demanda, solar e preço seguem modelos simulados.
- Sem previsão explícita: o estado apresenta apenas valores atuais e o horário, sem previsão das próximas horas.
- Degradação linear: baterias reais apresentam envelhecimento dependente de temperatura, profundidade de descarga e número de ciclos.
- Ausência de valor terminal: não existe bônus ou penalidade específica para o SOC final.
- Observabilidade parcial: fatores latentes, como o fator diário de nebulosidade, não aparecem diretamente na observação.
- Critério de sucesso relativo: a classificação depende da heurística escolhida e da tolerância de 20%.

Melhorias futuras:

- usar ações contínuas para controlar potência de carga e descarga;
- acrescentar previsões de demanda, geração solar e preço;
- testar cenários com rede limitada ou indisponível;
- modelar degradação não linear;
- avaliar diferentes capacidades de bateria;
- comparar recompensa com e sem custo de degradação;
- comparar preços fixos e variáveis;
- usar recurrent PPO ou redes com memória;
- aplicar otimização de hiperparâmetros;
- incluir intervalos de confiança e testes estatísticos;
- avaliar o agente em dados reais de demanda e geração.

14. Conclusão

O presente trabalho apresentou a modelagem e implementação de um ambiente de Aprendizagem por Reforço para o controle de uma bateria em uma microgrid com energia solar, utilizando a biblioteca Gymnasium. O problema foi formulado como um Processo de Decisão de Markov, no qual o agente deve decidir, a cada hora, entre manter, carregar, descarregar ou vender energia, considerando demanda, geração solar, preços da

energia e estado de carga da bateria.

Os experimentos realizados demonstraram que os algoritmos DQN e PPO foram capazes de aprender políticas relevantes para a operação da microgrid. Ambos superaram significativamente o agente aleatório e apresentaram redução do custo financeiro em comparação com a baseline heurística, evidenciando que a interação com o ambiente permitiu aos agentes desenvolver estratégias de tomada de decisão mais eficientes.

Entre os agentes de Aprendizagem por Reforço, o DQN apresentou o melhor equilíbrio entre economia financeira, estabilidade e taxa de sucesso, enquanto o PPO adotou uma política mais agressiva, com maior utilização da bateria e maior ocorrência de ações inválidas. A análise também mostrou que a heurística obteve recompensa total ligeiramente superior ao DQN devido ao menor número de ações inválidas e à menor degradação da bateria.

Dessa forma, o principal resultado do trabalho foi a identificação do trade-off entre redução de custo e preservação da bateria: utilizar a bateria com maior intensidade pode diminuir a compra de energia da rede, mas aumenta o desgaste do sistema de armazenamento. Conclui-se que a Aprendizagem por Reforço é uma abordagem promissora para o gerenciamento energético de microgrids, pois permite que o agente aprenda políticas de longo prazo capazes de equilibrar economia, confiabilidade operacional e vida útil da bateria.

15. Referências

<https://medium.com/datarisk-io/conceitos-para-enteder-reinforcement-learning-2b04d68bf4f9>

<https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae>

<https://jonathan-hui.medium.com/rl-proximal-policy-optimization-ppo-explained-77f014ec3f12>